

Colab (Practica)

Markdown



Introducción

[Markdown](#) es un lenguaje de marcado ligero que facilita el formato de texto de manera sencilla y legible. Fue creado en 2004 por John Gruber y Aaron Swartz. Su principal ventaja es que es fácil de escribir y leer, y se convierte fácilmente a HTML y otros formatos. Es ampliamente utilizado en documentación, blogs, y plataformas como GitHub para archivos README.

Elementos Básicos de Markdown

1. Encabezados

Crea encabezados con el símbolo `#`. Más `#` indican un encabezado de nivel inferior.

Ejemplo:

```
# Título Principal (H1)
## Subtítulo (H2)
### Sub-subtítulo (H3)
```

2. Párrafos

Escribe texto seguido de una línea en blanco para crear un nuevo párrafo.

Ejemplo:

Este es un párrafo simple.

Este es otro párrafo, separado del anterior por una línea en blanco.

3. Negrita y Cursiva

Destaca el texto usando negrita o cursiva:

- **Negrita:** Usa `**texto**` o `__texto__`.
- *Cursiva:* Usa `*texto*` o `_texto_`.

Ejemplo:

```
**Texto en negrita**  
*Texto en cursiva*
```

4. Listas

Crea listas ordenadas y no ordenadas fácilmente.

- **No ordenadas:** Usa `*`, `-`, o `+`.
- **Ordenadas:** Usa números seguidos de un punto.

Ejemplo:

```
- Manzanas  
- Naranjas  
  - Valencia  
  - Navel  
1. Primer paso  
2. Segundo paso  
3. Tercer paso
```

5. Enlaces

Inserta enlaces con la siguiente sintaxis:

Ejemplo:

```
[Visita Google](https://www.google.com)
```

6. Imágenes

Inserta imágenes de manera similar a los enlaces, pero precede con `!`.

Ejemplo:

```
![Logo de Markdown]  
(https://downloads.marketplace.jetbrains.com/files/18897/166369/icon/pluginIcon.png)
```

7. Citas

Destaca texto con citas utilizando `>` al principio de la línea.

Ejemplo:

> Esto es una cita importante.

8. Código en Python

Markdown permite resaltar código en línea y en bloques. Es ideal para compartir scripts de Python.

Ejemplo de código en línea:

El comando ``print("Hola, mundo")`` se utiliza para mostrar texto en Python.

Ejemplo de bloque de código:

```
def saludar(nombre):  
    return f"Hola, {nombre}!"
```

```
nombre = "John"  
print(saludar(nombre))
```

9. Fórmulas Matemáticas

Incorpora expresiones matemáticas utilizando LaTeX con *MathJax*.

Ejemplo de fórmula en línea:

La ecuación de Einstein es $E = mc^2$.

Ejemplo de fórmula en bloque:

```
$$  
\int_0^{\infty} \frac{\sin x}{x} dx = \frac{\pi}{2}  
$$
```

Python



Introducción

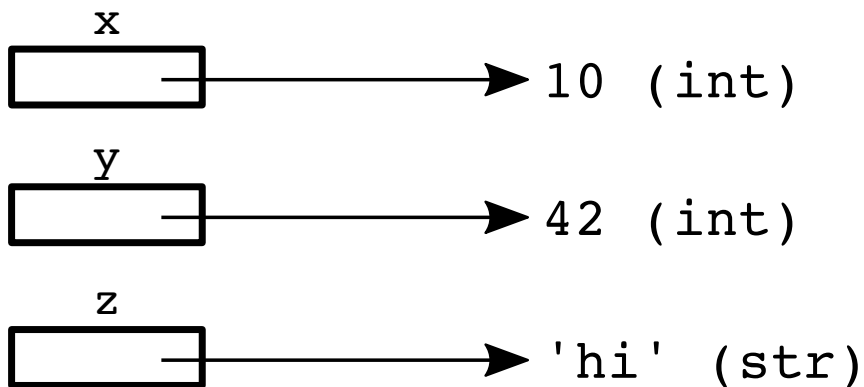
Python es un lenguaje de programación interpretado y de alto nivel. Fue creado por Guido van Rossum y lanzado por primera vez en 1991. Python se destaca por su legibilidad y sintaxis clara, lo

que lo hace ideal tanto para principiantes como para programadores experimentados.

Una de las características más destacadas de Python es su filosofía de diseño, que enfatiza la legibilidad del código y la facilidad de uso. El lenguaje utiliza una sintaxis limpia y utiliza el espaciado en blanco de manera significativa para estructurar el código, lo que lo hace fácil de leer y entender.

🔑 **Nota:** Para profundizar en conceptos básicos de Python, se recomienda estudiar del curso [Introducción a Python](#).

Nomenclatura



Sintaxis

Hola mundo!

Escribamos nuestro primer programa de Python, "¡Hola, mundo!". Es un programa simple que imprime Hello World! en el dispositivo de salida estándar (pantalla).

```
# imprimir "Hola Mundo!"  
print("Hola Mundo!");
```

Hola Mundo!

Variables

Las **variables** son contenedores que se utilizan para almacenar datos. Estos datos pueden ser números, cadenas de texto, listas, diccionarios, y más. En Python, no necesitas declarar el tipo de variable antes de asignarle un valor; el tipo de variable se determina automáticamente según el valor que se le asigne.

Además, las variables pueden contener una amplia variedad de tipos de datos, que determinan el tipo de valores que pueden almacenar y las operaciones que se pueden realizar con ellos.

```
# Variables numéricas  
numero_entero = 10  
numero_flotante = 3.14  
  
# Variables de cadena de texto
```

```

nombre = "Juan"
mensaje = 'Hola, mundo!'

# Variables booleanas
verdadero = True
falso = False

# Variables nulas
mi_variable_nula = None

# Imprimiendo los valores de las variables
print("Valor de numero_entero:", numero_entero)
print("Valor de numero_flotante:", numero_flotante)
print("Valor de nombre:", nombre)
print("Valor de mensaje:", mensaje)
print("Valor de verdadero:", verdadero)
print("Valor de falso:", falso)
print("Valor de mi_variable_nula:", mi_variable_nula)

```

```

Valor de numero_entero: 10
Valor de numero_flotante: 3.14
Valor de nombre: Juan
Valor de mensaje: Hola, mundo!
Valor de verdadero: True
Valor de falso: False
Valor de mi_variable_nula: None

```

Operaciones

Puedes realizar una variedad de operaciones en diferentes tipos de datos. Estas operaciones pueden ser aritméticas, de comparación, lógicas, de asignación, de pertenencia e identidad. A continuación, se describen los principales tipos de operaciones:

Operaciones Aritméticas

Las operaciones aritméticas se utilizan para realizar cálculos matemáticos simples. Algunas de las operaciones aritméticas básicas incluyen:

- Suma (+)
- Resta (-)
- Multiplicación (*)
- División (/)
- Potenciación (**)
- División entera (//)
- Módulo (%)

```

# Operaciones Aritméticas
a = 10
b = 3

suma = a + b
resta = a - b
multiplicacion = a * b
division = a / b
potenciacion = a ** b
division_entera = a // b

```

```

modulo = a % b

print("Operaciones Aritméticas:")
print("Suma:", suma)
print("Resta:", resta)
print("Multiplicación:", multiplicacion)
print("División:", division)
print("Potenciación:", potenciacion)
print("División entera:", division_entera)
print("Módulo:", modulo)

```

```

Operaciones Aritméticas:
Suma: 13
Resta: 7
Multiplicación: 30
División: 3.3333333333333335
Potenciación: 1000
División entera: 3
Módulo: 1

```

Operaciones de Comparación

Las operaciones de comparación se utilizan para comparar valores y devolver un valor booleano (Verdadero o Falso). Algunas operaciones de comparación comunes son:

- Igualdad (==)
- Desigualdad (!=)
- Mayor que (>)
- Menor que (<)
- Mayor o igual que (>=)
- Menor o igual que (<=)

```

# Operaciones de Comparación
igualdad = (a == b)
desigualdad = (a != b)
mayor_que = (a > b)
menor_que = (a < b)
mayor_igual = (a >= b)
menor_igual = (a <= b)

print("Operaciones de Comparación:")
print("Igualdad:", igualdad)
print("Desigualdad:", desigualdad)
print("Mayor que:", mayor_que)
print("Menor que:", menor_que)
print("Mayor o igual que:", mayor_igual)
print("Menor o igual que:", menor_igual)

```

```

Operaciones de Comparación:
Igualdad: False
Desigualdad: True
Mayor que: True
Menor que: False
Mayor o igual que: True
Menor o igual que: False

```

Operaciones Lógicas

Las operaciones lógicas se utilizan para realizar operaciones booleanas en valores booleanos. Algunas operaciones lógicas comunes incluyen:

- Y lógico (and)
- O lógico (or)
- Negación lógica (not)

```
# Operaciones Lógicas
verdadero = True
falso = False

y_logico = verdadero and falso
o_logico = verdadero or falso
negacion = not verdadero

print("Operaciones Lógicas:")
print("Y lógico:", y_logico)
print("O lógico:", o_logico)
print("Negación:", negacion)
```

```
Operaciones Lógicas:
Y lógico: False
O lógico: True
Negación: False
```

Operaciones de Asignación

Las operaciones de asignación se utilizan para asignar valores a variables. Algunas operaciones de asignación pueden combinar operaciones aritméticas con la asignación. Por ejemplo:

```
# Operaciones de Asignación
x = 5
x += 2 # Equivalente a x = x + 2

print("Operaciones de Asignación:")
print("Valor de x después de la asignación:", x)
```

```
Operaciones de Asignación:
Valor de x después de la asignación: 7
```

Operaciones con strings

Puedes realizar varias operaciones con cadenas de texto (strings) para manipular y trabajar con ellas de diversas formas. A continuación, se describen algunas operaciones comunes con ejemplos y resultados:

```
cadena1 = "Hola"
cadena2 = "mundo"

# Concatenación de cadenas
resultado_concatenacion = cadena1 + " " + cadena2
print("Concatenación:", resultado_concatenacion)
```

```
Concatenación: Hola mundo
```

```
cadena = "Python"
# Repetición de una cadena
```

```
resultado_repeticion = cadena * 3
print("Repetición:", resultado_repeticion)
```

Repetición: PythonPythonPython

```
# Indexación para obtener el primer carácter de la cadena
primer_caracter = cadena[0]
print("Indexación:", primer_caracter)
```

Indexación: P

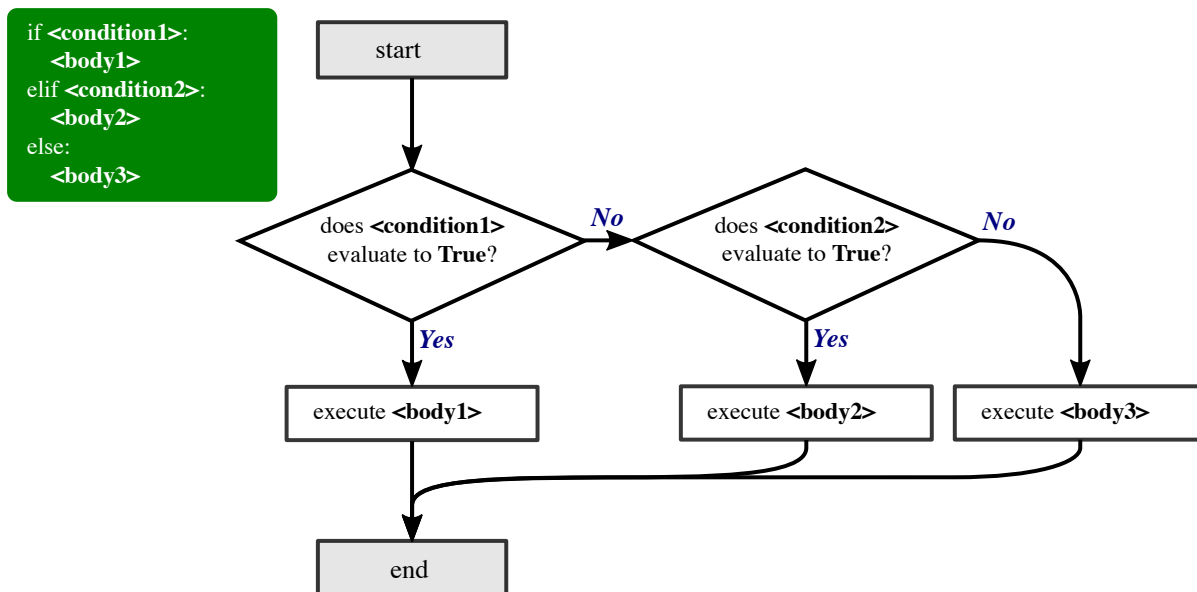
```
# Obtención de la longitud de la cadena
longitud = len(cadena)
print("Longitud:", longitud)
```

Longitud: 6

```
# Slicing para obtener una subcadena de la cadena original
subcadena = cadena[2:5]
print("Slicing:", subcadena)
```

Slicing: tho

Control de Flujo



Condicional if-elif-else

Los condicionales `if`, `elif` y `else` son estructuras fundamentales en Python que te permiten ejecutar diferentes bloques de código dependiendo de ciertas condiciones. A continuación se explica cómo funcionan:

Estructura básica

La estructura básica de un condicional `if` es la siguiente:

```
if condicion:
    # Código a ejecutar si la condición es verdadera
```

Si la condición es verdadera, se ejecuta el bloque de código indentado debajo de la declaración `if`. Si la condición es falsa, este bloque de código se omite.

Condicional if-else

Puedes agregar un bloque `else` para ejecutar un código alternativo cuando la condición en el `if` es falsa:

```
if condicion:
    # Código a ejecutar si la condición es verdadera
else:
    # Código a ejecutar si la condición es falsa
```

Condicional if-elif-else

Si tienes múltiples condiciones, puedes usar la estructura `elif` (abreviatura de "else if") para verificarlas en secuencia:

```
if condicion1:
    # Código a ejecutar si la condicion1 es verdadera
elif condicion2:
    # Código a ejecutar si la condicion2 es verdadera
else:
    # Código a ejecutar si ninguna de las condiciones anteriores es verdadera
```

Ejemplo

```
edad = 20

if edad < 18:
    print("Eres menor de edad")
elif edad >= 18 and edad < 65:
    print("Eres adulto")
else:
    print("Eres un adulto mayor")
```

Eres adulto

Bucle While

El bucle `while` es una estructura de control que permite repetir un bloque de código mientras una condición especificada sea verdadera. A continuación se explica cómo funciona:

Estructura básica

La estructura básica de un bucle `while` es la siguiente:

```
while condicion:
    # Código a repetir mientras la condición sea verdadera
```

El bloque de código indentado debajo del `while` se ejecutará repetidamente mientras la condición sea verdadera. Una vez que la condición se evalúa como falsa, la ejecución del bucle se detiene y el programa continúa con la siguiente instrucción después del bucle.

Ejemplo

```
contador = 0

while contador < 5:
```

```
print("El contador es:", contador)
contador += 1

print("¡Bucle completado!")
```

```
El contador es: 0
El contador es: 1
El contador es: 2
El contador es: 3
El contador es: 4
¡Bucle completado!
```

Ciclo For con la Función Range()

El ciclo `for` se utiliza para iterar sobre una secuencia (como una lista, una tupla, un diccionario, etc.) y realizar una acción en cada elemento de la secuencia. La función `range()` es comúnmente utilizada junto con un ciclo `for` para generar una secuencia de números en un rango específico. A continuación se explica cómo funciona:

Estructura básica

La estructura básica de un ciclo `for` con `range()` es la siguiente:

```
for variable in range(inicio, fin, paso):
    # Código a ejecutar en cada iteración
```

La función `range()` genera una secuencia de números que comienza desde el valor de "inicio" hasta el valor de "fin" (sin incluirlo), con un incremento definido por el valor de "paso". El bloque de código indentado debajo del `for` se ejecutará una vez para cada valor en la secuencia generada por `range()`.

Ejemplo

```
for i in range(5):
    print("El valor de i es:", i)

print("¡Ciclo completado!")
```

```
El valor de i es: 0
El valor de i es: 1
El valor de i es: 2
El valor de i es: 3
El valor de i es: 4
¡Ciclo completado!
```

Salir o Continuar un Ciclo

Puedes controlar el flujo de ejecución de un ciclo `for` o `while` utilizando las palabras clave `break` y `continue`. Estas palabras clave te permiten detener la ejecución del ciclo de forma prematura o saltar a la siguiente iteración del ciclo, respectivamente. A continuación se explica cómo funcionan:

Salir de un Ciclo con `break`

La palabra clave `break` se utiliza para salir de un ciclo de forma prematura si se cumple una cierta condición. Cuando se encuentra una instrucción `break`, el ciclo se detiene inmediatamente y la ejecución del programa continúa con la primera instrucción después del ciclo.

Ejemplo

```
# Ejemplo de salir de un ciclo con break

contador = 0

while True:
    contador += 1

    # Salir del ciclo si el contador llega a 5
    if contador == 5:
        print("¡El contador llegó a 5! Saliendo del ciclo.")
        break

    print("El valor del contador es:", contador)

print("¡Ciclo completado!")
```

```
El valor del contador es: 1
El valor del contador es: 2
El valor del contador es: 3
El valor del contador es: 4
¡El contador llegó a 5! Saliendo del ciclo.
¡Ciclo completado!
```

Continuar con la Siguiente Iteración con `continue`

La palabra clave `continue` se utiliza para saltar a la siguiente iteración del ciclo si se cumple una cierta condición. Cuando se encuentra una instrucción `continue`, el ciclo se detiene temporalmente y se pasa a la siguiente iteración del ciclo, omitiendo cualquier código restante dentro del bloque de ciclo para esa iteración en particular.

Ejemplo

```
# Ejemplo de continuar con la siguiente iteración con continue

contador = 0

while contador < 5:
    contador += 1

    # Saltar a la siguiente iteración si el contador es par
    if contador % 2 == 0:
        print("El contador es par. Continuando con la siguiente iteración.")
        continue

    print("El valor del contador es:", contador)

print("¡Ciclo completado!")
```

```
El valor del contador es: 1
El contador es par. Continuando con la siguiente iteración.
El valor del contador es: 3
El contador es par. Continuando con la siguiente iteración.
El valor del contador es: 5
¡Ciclo completado!
```

Estructura de datos

Data Structure	Ordered	Mutable	Constructor	Example
List	Yes	Yes	<code>[]</code> or <code>list()</code>	<code>[5.7, 4, 'yes', 5.7]</code>
Tuple	Yes	No	<code>()</code> or <code>tuple()</code>	<code>(5.7, 4, 'yes', 5.7)</code>
Set	No	Yes	<code>{}</code> * or <code>set()</code>	<code>{5.7, 4, 'yes'}</code>
Dictionary	No	Yes**	<code>{ }</code> or <code>dict()</code>	<code>{'Jun': 75, 'Jul': 89}</code>

Existen varias estructuras de datos integradas que te permiten almacenar y organizar colecciones de elementos de diferentes maneras. Algunas de las estructuras de datos más comunes son las listas, las tuplas, los conjuntos y los diccionarios. A continuación, se explican cada una de ellas con ejemplos y resultados impresos.

Listas

Una lista en Python es una colección ordenada y mutable de elementos. Puedes acceder a los elementos de una lista utilizando índices, y puedes modificar la lista agregando, eliminando o cambiando elementos.

```
# Crear una lista vacía
lista_vacia = []
print("Lista vacía:", lista_vacia)
```

```
Lista vacía: []
```

```
# Crear una lista con elementos de diferentes tipos
lista_mixta = [1, "dos", 3.0, True]
print("Lista mixta:", lista_mixta)
```

```
Lista mixta: [1, 'dos', 3.0, True]
```

```
# Acceder a elementos de una lista utilizando índices positivos y negativos
mi_lista = ['a', 'b', 'c', 'd', 'e']
print("Primer elemento:", mi_lista[0])
print("Último elemento:", mi_lista[-1])
```

```
Primer elemento: a
Último elemento: e
```

```
# Rebanado (slicing) de listas
print("Rebanado:", mi_lista[1:3])
```

Rebanado: ['b', 'c']

```
# Modificar elementos de una lista
mi_lista[0] = 'z'
print("Lista modificada:", mi_lista)
```

Lista modificada: ['z', 'b', 'c', 'd', 'e']

```
# Agregar elementos a una lista
mi_lista.append('f')
print("Lista después de agregar un elemento:", mi_lista)
```

Lista después de agregar un elemento: ['z', 'b', 'c', 'd', 'e', 'f']

```
# Extender una lista con otra lista
mi_otra_lista = ['g', 'h', 'i']
mi_lista.extend(mi_otra_lista)
print("Lista después de extenderla con otra lista:", mi_lista)
```

Lista después de extenderla con otra lista: ['z', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']

```
# Insertar un elemento en una posición específica
mi_lista.insert(2, 'nuevo')
print("Lista después de insertar un nuevo elemento:", mi_lista)
```

Lista después de insertar un nuevo elemento: ['z', 'b', 'nuevo', 'c', 'd', 'e', 'f', 'g', 'h', 'i']

```
# Eliminar un elemento de una lista
mi_lista.remove('c')
print("Lista después de eliminar un elemento:", mi_lista)
```

Lista después de eliminar un elemento: ['z', 'b', 'nuevo', 'd', 'e', 'f', 'g', 'h', 'i']

```
# Eliminar el último elemento de una lista y devolverlo
ultimo_elemento = mi_lista.pop()
print("Último elemento eliminado:", ultimo_elemento)
print("Lista después de eliminar el último elemento:", mi_lista)
```

Último elemento eliminado: i
Lista después de eliminar el último elemento: ['z', 'b', 'nuevo', 'd', 'e', 'f', 'g', 'h']

```
# Buscar el índice de un elemento en una lista
indice = mi_lista.index('e')
print("Índice del elemento 'e':", indice)
```

Índice del elemento 'e': 4

```
# Contar la cantidad de veces que aparece un elemento en una lista
cantidad = mi_lista.count('a')
print("Cantidad de veces que aparece 'a':", cantidad)
```

Cantidad de veces que aparece 'a': 0

```
# Revertir el orden de los elementos en una lista
mi_lista.reverse()
```

```
print("Lista después de revertir el orden:", mi_lista)
```

Lista después de revertir el orden: ['h', 'g', 'f', 'e', 'd', 'nuevo', 'b', 'z']

```
# Ordenar los elementos de una lista (si son del mismo tipo)
numeros = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
numeros.sort()
print("Lista de números ordenada:", numeros)
```

Lista de números ordenada: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

```
# Copiar una lista
copia_lista = mi_lista.copy()
print("Copia de la lista:", copia_lista)
```

Copia de la lista: ['h', 'g', 'f', 'e', 'd', 'nuevo', 'b', 'z']

```
# Vaciar una lista
mi_lista.clear()
print("Lista después de vaciarla:", mi_lista)
```

Lista después de vaciarla: []

Tuplas

Una tupla en Python es una colección ordenada e inmutable de elementos. Puedes acceder a los elementos de una tupla utilizando índices, pero no puedes modificar la tupla después de crearla.

```
# Crear una tupla vacía
tupla_vacia = ()
print("Tupla vacía:", tupla_vacia)
```

Tupla vacía: ()

```
# Crear una tupla con elementos de diferentes tipos
tupla_mixta = (1, "dos", 3.0, True)
print("Tupla mixta:", tupla_mixta)
```

Tupla mixta: (1, 'dos', 3.0, True)

```
# Acceder a elementos de una tupla utilizando índices positivos y negativos
mi_tupla = ('a', 'b', 'c', 'd', 'e')
print("Primer elemento:", mi_tupla[0])
print("Último elemento:", mi_tupla[-1])
```

Primer elemento: a
Último elemento: e

```
# Rebanado (slicing) de tuplas
print("Rebanado:", mi_tupla[1:3])
# No se puede modificar una tupla después de crearla
# mi_tupla[0] = 'z' # Esto producirá un error
```

Rebanado: ('b', 'c')

```
# Concatenar tuplas
otra_tupla = ('f', 'g')
concatenacion = mi_tupla + otra_tupla
print("Concatenación de tuplas:", concatenacion)
```

Concatenación de tuplas: ('a', 'b', 'c', 'd', 'e', 'f', 'g')

```
# Multiplicar una tupla
multiplicacion = otra_tupla * 3
print("Multiplicación de tupla:", multiplicacion)
```

Multiplicación de tupla: ('f', 'g', 'f', 'g', 'f', 'g')

```
# Obtener la longitud de una tupla
longitud = len(mi_tupla)
print("Longitud de la tupla:", longitud)
```

Longitud de la tupla: 5

```
# Buscar un elemento en una tupla
if 'c' in mi_tupla:
    print("El elemento 'c' está en la tupla.")
else:
    print("El elemento 'c' no está en la tupla.")
```

El elemento 'c' está en la tupla.

```
# Contar la cantidad de veces que aparece un elemento en una tupla
cantidad = mi_tupla.count('b')
print("Cantidad de veces que aparece 'b':", cantidad)
```

Cantidad de veces que aparece 'b': 1

```
# Encontrar el índice de un elemento en una tupla
indice = mi_tupla.index('d')
print("Índice del elemento 'd':", indice)
```

Índice del elemento 'd': 3

Conjuntos

Un conjunto en Python es una colección desordenada y mutable de elementos únicos. Puedes realizar operaciones de conjuntos como unión, intersección y diferencia entre conjuntos.

```
# Crear un conjunto vacío
conjunto_vacio = set()
print("Conjunto vacío:", conjunto_vacio)
```

Conjunto vacío: set()

```
# Crear un conjunto con elementos
mi_conjunto = {1, 2, 3, 4, 5}
print("Conjunto:", mi_conjunto)
```

Conjunto: {1, 2, 3, 4, 5}

```
# Agregar elementos a un conjunto
mi_conjunto.add(6)
print("Conjunto después de agregar un elemento:", mi_conjunto)
```

Conjunto después de agregar un elemento: {1, 2, 3, 4, 5, 6}

```
# Eliminar un elemento de un conjunto
mi_conjunto.remove(3)
print("Conjunto después de eliminar un elemento:", mi_conjunto)
```

Conjunto después de eliminar un elemento: {1, 2, 4, 5, 6}

```
# Verificar si un elemento está en un conjunto
if 4 in mi_conjunto:
    print("El elemento 4 está en el conjunto.")
else:
    print("El elemento 4 no está en el conjunto.")
```

El elemento 4 está en el conjunto.

```
# Realizar operaciones de conjunto
conjunto1 = {1, 2, 3, 4, 5}
conjunto2 = {4, 5, 6, 7, 8}
```

```
# Unión de conjuntos
union = conjunto1.union(conjunto2)
print("Unión de conjuntos:", union)
```

Unión de conjuntos: {1, 2, 3, 4, 5, 6, 7, 8}

```
# Intersección de conjuntos
interseccion = conjunto1.intersection(conjunto2)
print("Intersección de conjuntos:", interseccion)
```

Intersección de conjuntos: {4, 5}

```
# Diferencia entre conjuntos
diferencia = conjunto1.difference(conjunto2)
print("Diferencia entre conjuntos 1 y 2:", diferencia)
```

Diferencia entre conjuntos 1 y 2: {1, 2, 3}

```
# Comprobar si un conjunto es subconjunto de otro
if conjunto1.issubset(conjunto2):
    print("El conjunto 1 es un subconjunto del conjunto 2.")
else:
    print("El conjunto 1 no es un subconjunto del conjunto 2.")
```

El conjunto 1 no es un subconjunto del conjunto 2.

```
# Vaciar un conjunto
conjunto1.clear()
print("Conjunto 1 después de vaciarlo:", conjunto1)
```

Conjunto 1 después de vaciarlo: set()

Diccionarios

Un diccionario en Python es una colección desordenada y mutable de pares clave-valor. Puedes acceder a los valores utilizando las claves, y puedes agregar, modificar o eliminar elementos del diccionario.

```
# Crear un diccionario vacío
diccionario_vacio = {}
print("Diccionario vacío:", diccionario_vacio)
```

Diccionario vacío: {}

```
# Crear un diccionario con elementos
mi_diccionario = {"nombre": "Juan", "edad": 30, "ciudad": "Madrid"}
print("Diccionario:", mi_diccionario)
```


Diccionario: {'nombre': 'Juan', 'edad': 30, 'ciudad': 'Madrid'}

```
# Acceder a valores utilizando claves
print("Nombre:", mi_diccionario["nombre"])
print("Edad:", mi_diccionario["edad"])
```

Nombre: Juan
Edad: 30

```
# Modificar valores en un diccionario
mi_diccionario["edad"] = 31
print("Diccionario después de modificar la edad:", mi_diccionario)
```

Diccionario después de modificar la edad: {'nombre': 'Juan', 'edad': 31, 'ciudad': 'Madrid'}

```
# Agregar un nuevo par clave-valor
mi_diccionario["profesión"] = "Ingeniero"
print("Diccionario después de agregar una profesión:", mi_diccionario)
```

Diccionario después de agregar una profesión: {'nombre': 'Juan', 'edad': 31, 'ciudad': 'Madrid', 'profesión': 'Ingeniero'}

```
# Eliminar un par clave-valor
del mi_diccionario["ciudad"]
print("Diccionario después de eliminar la ciudad:", mi_diccionario)
```

Diccionario después de eliminar la ciudad: {'nombre': 'Juan', 'edad': 31, 'profesión': 'Ingeniero'}

```
# Verificar si una clave está en el diccionario
if "nombre" in mi_diccionario:
    print("La clave 'nombre' está en el diccionario.")
else:
    print("La clave 'nombre' no está en el diccionario.")
```

La clave 'nombre' está en el diccionario.

```
# Obtener todas las claves y valores del diccionario
claves = mi_diccionario.keys()
print("Claves del diccionario:", claves)
valores = mi_diccionario.values()
print("Valores del diccionario:", valores)
```

Claves del diccionario: dict_keys(['nombre', 'edad', 'profesión'])
Valores del diccionario: dict_values(['Juan', 31, 'Ingeniero'])

```
# Obtener pares clave-valor del diccionario como tuplas
items = mi_diccionario.items()
print("Pares clave-valor del diccionario:", items)
```

Pares clave-valor del diccionario: dict_items([('nombre', 'Juan'), ('edad', 31), ('profesión', 'Ingeniero')])

```
# Copiar un diccionario
copia_diccionario = mi_diccionario.copy()
print("Copia del diccionario:", copia_diccionario)
```

Copia del diccionario: {'nombre': 'Juan', 'edad': 31, 'profesión': 'Ingeniero'}

```
# Vaciar un diccionario
mi_diccionario.clear()
```

```
print("Diccionario después de vaciarlo:", mi_diccionario)
```

```
Diccionario después de vaciarlo: {}
```

Funciones

Diagrama de la sintaxis de una función en Python:

```
def function_name(args):  
    #code in function
```

Los números indican las partes de la sintaxis:

- 1: `def`
- 2: `function_name`
- 3: `(args)`
- 4: `:`
- 5: `#code in function`

Definiciones

Una función es un bloque de código que realiza una tarea específica y puede ser llamado desde cualquier parte del programa. Las funciones pueden tener parámetros (datos que se pasan a la función) y pueden devolver un resultado. Al utilizar funciones, podemos dividir el código en partes más pequeñas y manejables, lo que facilita la comprensión y la depuración.

Sintaxis

La sintaxis básica de una función en Python es la siguiente:

```
def nombre_funcion(parametros):  
    # Cuerpo de la función  
    # Realizar operaciones  
    return resultado
```

- **def:** Es una palabra clave que indica que se está definiendo una función.
- **nombre_funcion:** Es el nombre que le damos a la función. Debe seguir las mismas reglas de nomenclatura que las variables.
- **parámetros:** Son los datos que la función espera recibir. Pueden ser opcionales.
- **return:** Es una palabra clave que indica el valor que la función devolverá. Puede devolver cero o más valores.

```
# Definir una función sin parámetros y sin valor de retorno  
def saludar():  
    print("¡Hola!")
```

```
# Llamar a la función  
saludar()
```

```
¡Hola!
```

```
# Definir una función con parámetros y sin valor de retorno  
def saludar_persona(nombre):  
    print("¡Hola,", nombre, "!")
```

```
# Llamar a la función con un argumento  
saludar_persona("Juan")
```

iHola, Juan !

```
# Definir una función con parámetros y con valor de retorno
def sumar(a, b):
    return a + b

# Llamar a la función y almacenar el resultado en una variable
resultado = sumar(3, 5)
print("Resultado de la suma:", resultado)
```

Resultado de la suma: 8

```
# Función con parámetros con valores predeterminados
def multiplicar(a, b=2):
    return a * b

# Llamar a la función con un solo argumento
resultado1 = multiplicar(3)
print("Resultado de la multiplicación (con valor predeterminado):", resultado1)

# Llamar a la función con dos argumentos
resultado2 = multiplicar(3, 4)
print("Resultado de la multiplicación (con argumento específico):", resultado2)
```

Resultado de la multiplicación (con valor predeterminado): 6
Resultado de la multiplicación (con argumento específico): 12

```
# Función con un número variable de argumentos
def promedio(*args):
    return sum(args) / len(args)

# Llamar a la función con diferentes cantidades de argumentos
resultado1 = promedio(2, 4, 6)
print("Promedio de 2, 4 y 6:", resultado1)

resultado2 = promedio(1, 3, 5, 7, 9)
print("Promedio de 1, 3, 5, 7 y 9:", resultado2)
```

Promedio de 2, 4 y 6: 4.0
Promedio de 1, 3, 5, 7 y 9: 5.0

```
# Función con argumentos de palabra clave
def mostrar_info(nombre, edad):
    print("Nombre:", nombre)
    print("Edad:", edad)

# Llamar a la función utilizando argumentos de palabra clave
mostrar_info(nombre="Ana", edad=25)
```

Nombre: Ana
Edad: 25

```
# Función con retorno de múltiples valores
def dividir(a, b):
    cociente = a // b
    resto = a % b
    return cociente, resto

# Llamar a la función y almacenar los valores de retorno en variables separadas
cociente, resto = dividir(10, 3)
```

```
print("Cociente:", cociente)
print("Resto:", resto)
```

Cociente: 3
Resto: 1

```
# Funciones anidadas
def exterior():
    print("Función exterior")

    def interior():
        print("Función interior")

    interior()

# Llamar a la función exterior, que también llama a la función interior
exterior()
```

Función exterior
Función interior

Excepciones

The diagram illustrates a Python SyntaxError. At the top, the error message is shown: "File `\"/home/ubuntu/workspace/sols/test.py\"`, line 2". A green arrow labeled '1' points to the file path, and a blue arrow labeled '2' points to "line 2". Below this, the code snippet `print "there"` is shown with a red arrow labeled '3' pointing to the closing quote of the string. Underneath the code, the full error message is displayed: `SyntaxError: Missing parentheses in call to 'print'`. A pink arrow labeled '4' points to the `SyntaxError:` prefix, and a cyan arrow labeled '4' points to the descriptive text `Missing parentheses in call to 'print'`.

En Python, los errores se dividen principalmente en tres categorías: errores de sintaxis, excepciones y errores semánticos. Cada uno de estos tipos de errores tiene sus propias características y se manejan de manera diferente en el código.

Errores de Sintaxis

Los errores de sintaxis en Python ocurren cuando el código no sigue las reglas gramaticales del lenguaje. Estos errores se detectan durante la fase de análisis del código y generalmente provocan que el intérprete de Python detenga la ejecución del programa y muestre un mensaje de error que indica la ubicación y la naturaleza del error. Aquí hay una lista de algunos errores de sintaxis comunes y ejemplos de cómo corregirlos:

Falta de paréntesis, corchetes o llaves

```
# Error de sintaxis: Falta un paréntesis de cierre
print("Hola mundo"
```

```
Cell In[73], line 2
    print("Hola mundo"
      ^
```

SyntaxError: unexpected EOF while parsing

Falta de comillas

```
# Error de sintaxis: Falta una comilla de cierre
mensaje = "Hola mundo
```

```
Cell In[74], line 2
    mensaje = "Hola mundo
      ^
```

SyntaxError: EOL while scanning string literal

Falta de dos puntos

```
# Error de sintaxis: Falta dos puntos al final de la declaración if
if True
    print("Es verdadero")
# Solución: Agregar dos puntos al final de la declaración if
if True:
    print("Es verdadero")
```

```
Cell In[75], line 2
    if True
      ^
```

SyntaxError: invalid syntax

Indentación incorrecta

```
# Error de sintaxis: Declaración indentada incorrectamente
if True:
print("Es verdadero")
```

```
Cell In[76], line 3
    print("Es verdadero")
      ^
```

IndentationError: expected an indented block

Uso incorrecto de palabras clave

```
# Error de sintaxis: Uso incorrecto de la palabra clave 'else'
if True:
    print("Es verdadero")
elsee:
    print("Es falso")
```

```
Cell In[77], line 4
    elsee:
      ^
```

SyntaxError: invalid syntax

Excepciones

Las excepciones en Python son eventos que ocurren durante la ejecución del programa y pueden alterar el flujo normal de ejecución. Cuando una excepción se produce, Python detiene la ejecución

del programa y busca un bloque de código que pueda manejar la excepción. Aquí se presentan algunos tipos comunes de excepciones en Python y cómo manejarlos:

Excepción de tipo `SyntaxError`

```
# Error de sintaxis: Falta un paréntesis de cierre
print("Hola mundo"
```

```
Cell In[78], line 2
    print("Hola mundo"
```

SyntaxError: unexpected EOF while parsing

Excepción de tipo `NameError`

```
# Error de nombre: Variable no definida
print(variable_no_definida)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[79], line 2
      1 # Error de nombre: Variable no definida
----> 2 print(variable_no_definida)
```

NameError: name 'variable_no_definida' is not defined

Excepción de tipo `TypeError`

```
# Error de tipo: Suma entre tipos incompatibles
resultado = 10 + "20"
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[80], line 2
      1 # Error de tipo: Suma entre tipos incompatibles
----> 2 resultado = 10 + "20"
```

TypeError: unsupported operand type(s) for +: 'int' and 'str'

Excepción de tipo `ValueError`

```
# Error de valor: Argumento fuera de rango
import math
raiz_negativa = math.sqrt(-9)
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[81], line 3
      1 # Error de valor: Argumento fuera de rango
      2 import math
----> 3 raiz_negativa = math.sqrt(-9)
```

ValueError: math domain error

Excepción de tipo `ZeroDivisionError`

```
# Error de división por cero
division = 10 / 0
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[82], line 2
      1 # Error de división por cero
----> 2 division = 10 / 0

ZeroDivisionError: division by zero
```

Excepción de tipo `OverflowError`

```
# Error de desbordamiento: Número demasiado grande para ser representado
numero_grande = 20.0 ** 20.0 ** 20.0
```

```
-----
OverflowError                                    Traceback (most recent call last)
Cell In[83], line 2
      1 # Error de desbordamiento: Número demasiado grande para ser representado
----> 2 numero_grande = 20.0 ** 20.0 ** 20.0

OverflowError: (34, 'Result too large')
```

Manejo de Excepciones

```
try:
    # Código propenso a generar una excepción
    resultado = 10 / 0
except ZeroDivisionError:
    # Manejar la excepción
    print("Error: División por cero")
```

```
Error: División por cero
```

Errores Semánticos

Los errores semánticos en Python son más difíciles de detectar que los errores de sintaxis o las excepciones porque el código puede ejecutarse sin generar mensajes de error, pero produce resultados inesperados o incorrectos debido a una lógica incorrecta en el programa. Aquí se presentan algunos ejemplos de errores semánticos comunes en Python:

Operaciones incorrectas

```
# Error semántico: Operación incorrecta
resultado = 10 / 0 # División por cero
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[85], line 2
      1 # Error semántico: Operación incorrecta
----> 2 resultado = 10 / 0 # División por cero

ZeroDivisionError: division by zero
```

Lógica incorrecta

```
# Error semántico: Lógica incorrecta
def calcular_promedio(valores):
    suma = 0
    for valor in valores:
        suma += valor
    promedio = suma / len(valores)
    return promedio

# Llamar a la función con una lista vacía
valores = []
promedio = calcular_promedio(valores)
print("El promedio es:", promedio)
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[86], line 11
      9 # Llamar a la función con una lista vacía
     10 valores = []
----> 11 promedio = calcular_promedio(valores)
     12 print("El promedio es:", promedio)

Cell In[86], line 6, in calcular_promedio(valores)
      4 for valor in valores:
      5     suma += valor
----> 6 promedio = suma / len(valores)
      7 return promedio

ZeroDivisionError: division by zero
```

En este caso, si la lista `valores` está vacía, se generará un error al intentar dividir por cero, lo cual es un error semántico en el contexto del programa.

Uso incorrecto de estructuras de datos

```
# Error semántico: Uso incorrecto de una lista
numeros = [1, 2, 3, 4, 5]
ultimo_numero = numeros[10] # Acceso a un índice fuera de rango
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[87], line 3
      1 # Error semántico: Uso incorrecto de una lista
      2 numeros = [1, 2, 3, 4, 5]
----> 3 ultimo_numero = numeros[10] # Acceso a un índice fuera de rango

IndexError: list index out of range
```

Valores incorrectos

```
# Error semántico: Asignación incorrecta de valores
edad = -10 # Edad negativa
```

Lógica de negocio incorrecta

```
# Error semántico: Cálculo incorrecto
precio = 100
```



```
descuento = 200 # Descuento mayor que el precio
precio_final = precio - descuento
```

Cómo Identificar Errores Semánticos

Los errores semánticos generalmente se detectan cuando el programa produce resultados inesperados o incorrectos. Para identificar y corregir estos errores, es importante comprender la lógica del programa y revisar cuidadosamente el código en busca de posibles problemas. La depuración paso a paso y las pruebas exhaustivas del programa pueden ayudar a identificar y corregir los errores semánticos de manera efectiva.